

VELLORE INSTITUTE OF TECHNOLOGY BHOPAL

VITYARTHI PROJECT

JAVA CORE APPLICATION REPORT

Student Grade & Attendance Tracker

A Modular Object-Oriented System for Academic Continuous Assessment,
Defaulter Tracking, and Automated File Persistence

COURSE TITLE

Programming in Java

COURSE CODE

CSE2005

STUDENT NAME

Atharv Vijayvargiya

REGISTRATION NUMBER

25BAI10420

FACULTY MENTOR

Nilesh Kunhare

GITHUB REPOSITORY

github.com/th25thbam/StudentTracker

01 Introduction & Motivation

In academic institutions, monitoring student performance across continuous internal assessments (quizzes, lab assignments, tests) and maintaining mandatory attendance records represent critical administrative responsibilities. Frequently, educators and teaching assistants handle these duties using disparate spreadsheets, manual tally sheets, or cumbersome institutional enterprise software. This fragmentation often causes computational oversights, lost historical data, and delays in identifying underperforming students.

The **Student Grade & Attendance Tracker** is developed as a lightweight, reliable, console-based Java application. It encapsulates object-oriented programming (OOP) principles to provide a structured CLI interface that computes cumulative score averages, monitors real-time class attendance percentages, flags statutory defaulters (< 75%), and serializes state directly into a local flat-file storage system.

02 Problem Statement

Core Challenge

Instructors face significant overhead in calculating real-time averages and enforcing the 75% attendance threshold across large cohorts. Manual data entry lacks instant boundary validation, resulting in erroneous grade captures and delayed academic intervention for at-risk students.

03 Functional Requirements

Module 1: Student Record Registry

Allows instructors to register students with a distinct alphanumeric Student ID, full name, and raw session metrics (classes attended and total sessions held).

Module 2: Score Logging & Validation

Appends assessment scores to specific student profiles while executing boundary checks (0.0 to 100.0), triggering custom exceptions on violations.

Module 3: Analytics & Defaulter Engine

Calculates cumulative percentage averages and determines mandatory attendance compliance, tagging each record as [ELIGIBLE] or [DEFICIT (<75%)].

Module 4: Persistence Management

Performs file deserialization during bootstrap and automatic comma-delimited output serialization on state mutation into `students.txt`.

04 Non-Functional Requirements

- **Reliability:** Guarantees transactional state writing on every addition. Application restarts do not forfeit recorded continuous assessment scores.
- **Robust Error Handling:** Replaces abrupt terminal crashes with safe `try-catch` blocks for numeric parsing, missing files, and custom score constraints.
- **Maintainability:** Modular architecture featuring separate packages for entities (`model`), business logic (`app`), storage (`storage`), and errors (`exception`).
- **Usability:** Fast, predictable CLI loop driven by clear numerical menus, input prompts, and organized tabulation summaries.

05 System Architecture

StudentTracker utilizes a three-tier modular pattern decoupling the user presentation loop, operational service controller, domain data definitions, and persistence mechanisms.

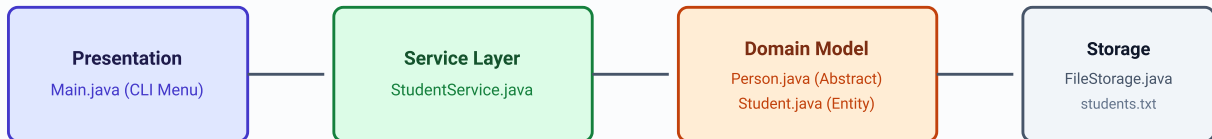


Figure 5.1: Multi-Tier Modular Component Architecture of StudentTracker

06 Design Diagrams: Class & Use Case Structure

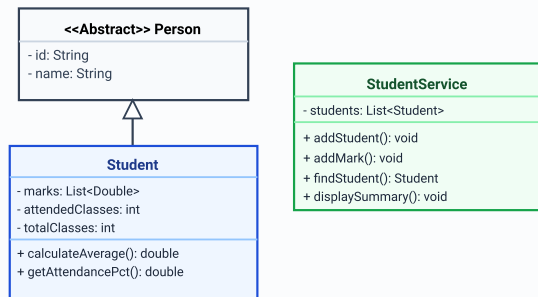


Figure 6.1: UML Class Diagram

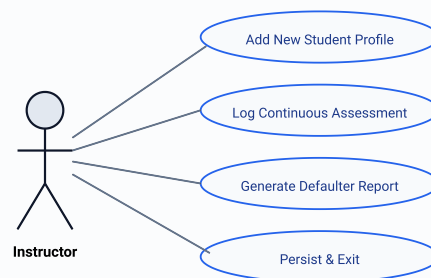
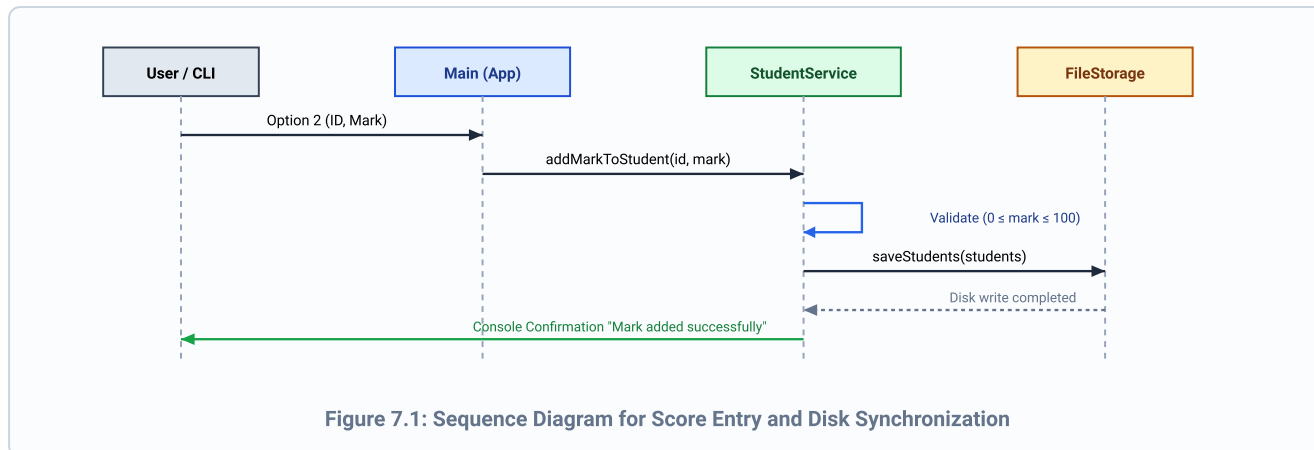


Figure 6.2: Use Case Diagram

07 Design Diagrams: Sequence & Workflow Logic

The interaction sequence models transactional operations: when input is captured by Main, validated through custom constraints, saved in memory via StudentService, and flushed directly to disk using FileStorage.



08 Design Decisions & Implementation Rationale

Design Consideration	Selected Approach	Technical Rationale
Persistence Strategy	Comma-delimited flat file (students.txt)	Zero setup overhead. Eliminates external SQL installation constraints while satisfying offline evaluation needs.
Domain Polymorphism	Abstract Person class base	Demonstrates object-oriented extensibility. Enables future system scaling to include Faculty or Advisor classes.
Exception Handling	Custom InvalidScoreException	Enforces checked business domain rules at compile-time instead of relying on generic runtime errors.
Collections Data Structure	ArrayList<Double> for marks	Allows dynamic appending of non-fixed test scores per student without memory overallocation.

09 Implementation Details

The project is implemented in **Core Java (JDK 17+)** conforming to standard package directory mappings. Key mechanisms include:

- **Stream-Safe File Parsing:** `FileStorage.java` uses `BufferedReader` within a `try-with-resources` block, automatically handling file closure and skipping corrupted lines.
- **Encapsulated Domain Math:** Calculation of aggregate averages and attendance rates is encapsulated directly inside `Student.java`, maintaining high cohesion.

10 Execution Results & Screenshots

```
● atharv@handler1:~/developer/StudentTracker$ javac -d bin model/*.java exception/*.java storage/*.java app/*.java
○ atharv@handler1:~/developer/StudentTracker$ java -cp bin app.Main
```

```
=== STUDENT TRACKER MENU ===
1. Add New Student
2. Enter Marks for Student
3. Display Summary Report
4. Exit
Enter choice (1-4): 1
Enter Student ID: 101
Enter Name: arthur
Enter Attended Classes: 10
Enter Total Classes: 11
Student recorded.
```

```
=== STUDENT TRACKER MENU ===
1. Add New Student
2. Enter Marks for Student
3. Display Summary Report
4. Exit
Enter choice (1-4): 2
Enter Student ID: 101
Enter Mark (0-100): 101
Error: Score must be between 0 and 100.
```

```
=== STUDENT TRACKER MENU ===
1. Add New Student
2. Enter Marks for Student
3. Display Summary Report
4. Exit
Enter choice (1-4): 3
```

```
--- Performance & Defaulter Summary ---
```

ID: 101	Name: arthur	Average: 0.00	Attendance: 90.9% [ELIGIBLE]
ID: 101	Name: arthur	Average: 0.00	Attendance: 90.9% [ELIGIBLE]
ID: 101	Name: arthur	Average: 0.00	Attendance: -100.0% [DEFICIT (<75%)]

11 Testing Approach

Test Case	Input Stimulus	Expected Behavior	Status
TC-01: Boundary Score Check	Mark = 105.0	Throws <code>InvalidScoreException</code> ; shows "Score must be between 0 and 100"	PASS
TC-02: Defaulter Flagging	Attended: 18 / Total: 30 (60%)	Calculates 60.0% attendance; assigns DEFICIT (<75%) flag	PASS
TC-03: Non-numeric Input	Mark = "eighty"	Catches <code>NumberFormatException</code> ; prompts "Input must be a valid number"	PASS
TC-04: Persistence Reload	Relaunch console process	Reads <code>students.txt</code> ; populates student list without data loss	PASS

12 Challenges Faced & Solutions

- **Parsing Complex Tokens:** Parsing semi-colon separated score arrays within a comma-separated record occasionally yielded array index errors on empty marks. *Resolution:* Implemented defensive array bounds checking and blank token filtering.
- **CLI Scanner Skipping:** Using `Scanner.nextInt()` prior to `nextLine()` caused carriage return skips. *Resolution:* Standardized all CLI reading to `nextLine().trim()` with explicit type conversions.

13 Learnings & Key Takeaways

This project reinforced core object-oriented principles, particularly the value of abstract class inheritance for extensible entity design and defensive programming through custom exceptions. Implementing flat-file serialization also provided practical insights into data parsing and exception recovery.

14 Future Enhancements

- **Relational DB Migration:** Transition from local text files to SQLite/PostgreSQL using JDBC for query indexing.
- **Graphical UI:** Implement a JavaFX desktop interface with visual attendance charts.
- **Weighted Grading:** Support percentage weights for quizzes, assignments, and terminal exams.

15 References

1. Oracle Corp., *Java Platform, Standard Edition Documentation (JDK 17 API Specification)*, 2023.
2. Schildt, H., *Java: The Complete Reference*, 12th Edition, McGraw Hill, 2021.
3. Martin, R. C., *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall, 2008.